

CareerConnect - Student Mentor and Recruiter Portal Project Report

By

**Chirag Pawar - AF05275833
Dhiraj Hindurao - AF05251523**

Index

Sr.no	Topic	Page no
1	Title of Project	1
2	Acknowledgement	3
3	Abstract	4
4	Introduction	5
5	Objective	6
6	System Analysis	7
7	System Design	22
8	Screenshots / Interface Views	32
9	Coding	39
10	Testing	46
11	Report	47
12	Future Scope	48
13	Conclusion	48
14	Bibliography	49
15	References	49

Acknowledgement

The project “CareerConnect – Student Mentor and Recruiter Portal” is the project work carried out by

Name	Enrollment No
Chirag Pawar	AF05275833
Dhiraj Hindurao	AF05251523

Under the Guidance.

We are thankful to our project guide for guiding us to complete the project. The suggestions, technical direction, and valuable information regarding the formation of the project report helped us in completing the project and its related work.

We are also thankful to our family members and friends who supported us throughout the development, testing, and documentation of the CareerConnect system.

Abstract

CareerConnect is a web-based career networking and recruitment platform developed using the MERN stack. The system connects students with mentors or experienced professionals and also provides a structured space for recruiters or job providers to publish opportunities and manage candidates. The platform brings mentorship, career guidance, job applications, candidate shortlisting, and interview scheduling into one application.

Students can create profiles containing education, skills, interests, experience, and career information. They can search for mentors, view mentor profiles, request mentorship or guidance, communicate through messages, request sessions, provide feedback, and apply for suitable jobs. Mentors can maintain professional profiles, respond to mentorship requests, conduct guidance sessions, and view job interview schedules when selected as candidates.

Recruiters can create and manage job posts, view applicants, review candidate profiles, shortlist candidates, and manually schedule online interviews using a Google Meet or other valid meeting link. The system stores interview date, candidate, recruiter, job, meeting link, message, and interview status so that candidates can access scheduled interviews from their dashboard.

The project focuses on role-based access, organized career information, simple navigation, reusable components, responsive pages, and secure communication between frontend, backend, and MongoDB.

1. Introduction

In today's digital environment, students and professionals need reliable ways to discover career opportunities, receive guidance, and connect with people who have practical industry experience. At the same time, recruiters need a convenient platform for publishing jobs and managing applications. Traditional communication through separate channels can make these activities difficult to organize.

CareerConnect is designed as an integrated web application that brings students, mentors, and recruiters together. Students can maintain a professional profile, search for mentors, request guidance, book sessions, explore jobs, and apply for positions. Mentors can present their expertise and support students through mentorship activities. Recruiters can publish jobs, review applications, shortlist candidates, and schedule online interviews.

The application follows a MERN architecture consisting of MongoDB for data storage, Express.js and Node.js for backend services, and React for the frontend. The separation between frontend and backend allows the system to provide reusable user interfaces and API-based communication.

1.1 Need for the Present Work

The project is needed to reduce the fragmentation between learning, mentorship, and recruitment activities. A single platform can organize profiles, mentorship requests, sessions, reviews, jobs, applications, and interviews in a structured manner.

1.2 Objective of the Present Work

- To develop a web application that connects students, mentors/professionals, and recruiters.
- To provide students with a structured profile for education, skills, interests, and experience.
- To allow students to search for mentors and request mentorship or career guidance.
- To provide mentorship sessions and feedback with a 1–7 rating mechanism.
- To allow recruiters to publish job opportunities and manage applicants.
- To support candidate shortlisting and hiring-related workflows.
- To allow recruiters to schedule future online interviews with meeting links.
- To provide role-based dashboards so each user sees relevant functions.
- To store application data securely in MongoDB and communicate through REST APIs.
- To provide a simple, responsive, and maintainable interface.

3. System Analysis

3.1 PROBLEM DEFINITION

Students often use different platforms for learning guidance, mentorship, job searching, and applications. Mentors may not have a structured method for receiving and managing guidance requests. Recruiters similarly need a consistent workflow for posting jobs, reviewing candidates, shortlisting applicants, and arranging interviews. CareerConnect addresses these issues by combining the three participant groups in one system.

3.2 Preliminary Investigation

- Student: manages profile, finds mentors, requests mentorship, communicates, attends sessions, gives feedback, searches jobs, and applies.
- Mentor/Professional: maintains expertise profile, handles mentorship requests, manages sessions, communicates, and can participate in recruitment.
- Recruiter/Job Provider: posts jobs, views applicants, shortlists candidates, and schedules interviews.
- System: maintains role-specific dashboards, notifications, messages, applications, reviews, and interview records.

3.3 Feasibility Study

The feasibility study evaluates whether CareerConnect can be implemented using available technologies and resources.

Technical Feasibility

- React provides component-based frontend development.
- Node.js and Express.js provide REST API services.
- MongoDB provides flexible document-based storage for users, jobs, applications, mentorship, and interviews.
- Axios is used for frontend API communication.
- React Router supports navigation between dashboards and pages.

Economic Feasibility

- The project uses widely available development tools and open-source technologies.
- The system can be developed and tested on a standard personal computer.
- A local MongoDB database can be used during development without additional database hosting costs.

Operational Feasibility

- Role-based dashboards make the system easier to operate.
- Forms and profile pages provide structured data entry.
- The application supports practical workflows for mentorship and recruitment.

3.4 Project Planning

Project planning defines the stages used to build CareerConnect in a structured way.

- 1. Preliminary Investigation – identify the need for a unified career platform.
- 2. Requirement Analysis – identify student, mentor, and recruiter workflows.
- 3. System Design – plan modules, database collections, routes, and UI.
- 4. Frontend Development – implement React components and dashboards.
- 5. Backend Development – implement Node.js/Express routes and MongoDB models.
- 6. Integration – connect React pages to REST APIs.
- 7. Testing – test authentication, profiles, mentorship, jobs, applications, and interviews.
- 8. Deployment / Maintenance – prepare the application for future hosting and enhancements.

3.5 Software Requirement Specification (SRS)

System Overview

CareerConnect is organized into three primary role modules: Student, Mentor/Professional, and Recruiter/Job Provider. Common supporting modules include authentication, profile management, messaging, notifications, jobs, applications, mentorship, reviews, sessions, and interviews.

Module	Main Functions
Authentication	Register, login, role selection, session/local storage handling
Student	Profile, mentors, mentorship, sessions, jobs, applications, interviews
Mentor	Professional profile, mentorship requests, sessions, messages, interviews
Recruiter	Jobs, applicants, shortlisting, candidate review, interviews
Communication	Messages and notifications
Reviews	Feedback and 1-7 rating for mentorship activities
Interview	Future date/time, meeting link, message, scheduled status

3.6 Software & Hardware Requirements

Software Requirements

- Frontend: React with Vite.
- Backend: Node.js with Express.js.
- Database: MongoDB.
- API Communication: Axios.
- Routing: React Router DOM.
- Development Environment: Visual Studio Code.
- Browser: Google Chrome or another modern browser.
- Database Tool: MongoDB Compass / MongoDB Shell.

Hardware Requirements

- Processor: Intel i5 / equivalent or higher recommended.
- RAM: Minimum 8 GB recommended.
- Storage: At least 10 GB free for development, dependencies, and database.
- Connectivity: Internet access for package installation and online meeting links.

3.7 Functional Requirements

- Users can register and log in according to their role.
- Users can maintain and update profile information.
- Students can search mentors and request mentorship.
- Mentors can manage mentorship requests and sessions.
- Recruiters can create and manage job posts.
- Applicants can apply for available jobs.
- Recruiters can view and shortlist applicants.
- Recruiters can schedule online interviews for selected candidates.
- Candidates can view scheduled interviews and use the meeting link.
- Users can exchange messages and receive notifications.

3.8 Non-Functional Requirements

- Usability – the interface should be understandable for students, mentors, and recruiters.
- Performance – API requests should return data efficiently for normal project-scale usage.
- Security – authentication and role checks should prevent unauthorized operations.
- Maintainability – React components, Express routes, and MongoDB models are separated into modules.
- Scalability – the architecture can be extended with additional modules and hosted services.
- Reliability – validation is applied to important forms and interview scheduling.
- Responsiveness – pages should remain usable across common desktop and mobile screen sizes.

3.9 Role-Based Access

Feature	Student	Mentor	Recruiter
Profile	✓	✓	✓
Search Mentors	✓	—	—
Mentorship Requests	✓	✓	—
Sessions	✓	✓	—
Jobs	View / Apply	View / Apply	Post / Manage
Applicants	—	—	✓
Shortlisting	—	—	✓
Interviews	View / Join	View / Join	Schedule
Messages	✓	✓	✓
Notifications	✓	✓	✓

3.10 Development Paradigm

Development Model: Adapted Waterfall with Iterative Feedback

The project follows a structured development sequence while allowing feedback during implementation and testing. Requirements are identified first, followed by design, coding, integration, testing, and maintenance. When an issue is discovered during testing, the related module can be refined without changing unrelated functionality.

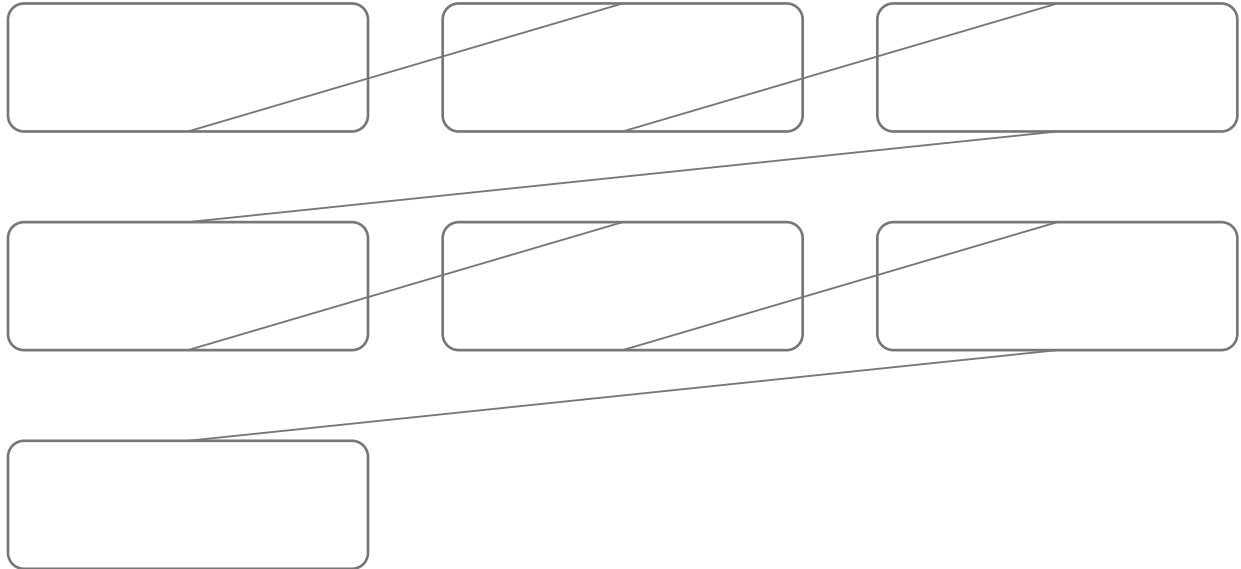


Fig. 3.1: Adapted development process.

3.11 Data Flow Diagram

A Data Flow Diagram (DFD) represents how information moves between users, the CareerConnect application, APIs, and the MongoDB database. The central system receives profile, mentorship, job, application, message, and interview data and returns the corresponding results to users.

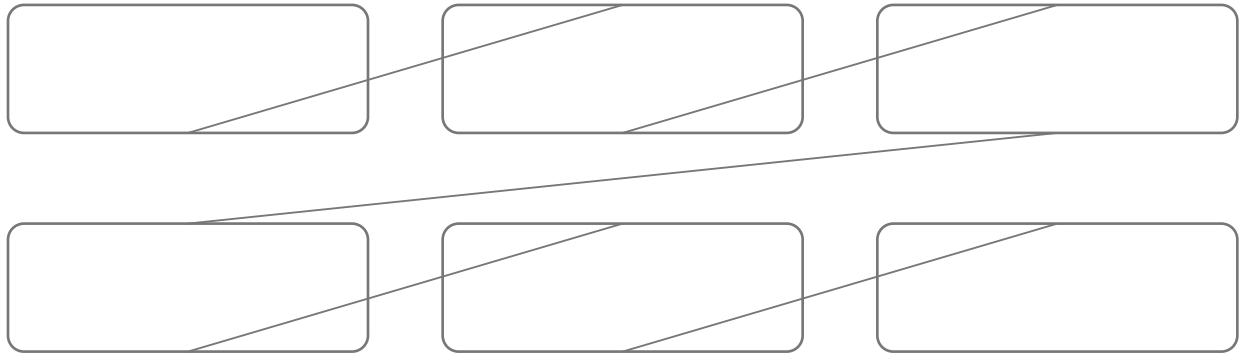


Fig. 3.2: Level-0 DFD representation.

3.12 First-Level DFD

The first-level DFD divides CareerConnect into major processes. Authentication handles registration and login. Profile Management handles user information. Mentorship handles requests and sessions. Job Management handles job posts and applications. Recruitment handles shortlisting and interviews. Communication handles messages and notifications.

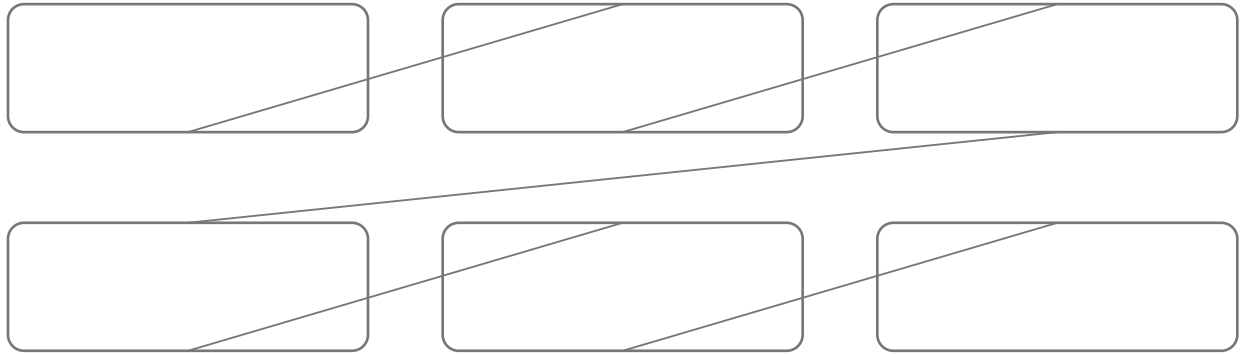


Fig. 3.3: Major functional processes.

3.13 Student DFD

The Student workflow begins with login and profile retrieval. The student can search mentors, view mentor details, send mentorship requests, request sessions, communicate through messages, give feedback, browse jobs, apply for jobs, and view scheduled interviews.

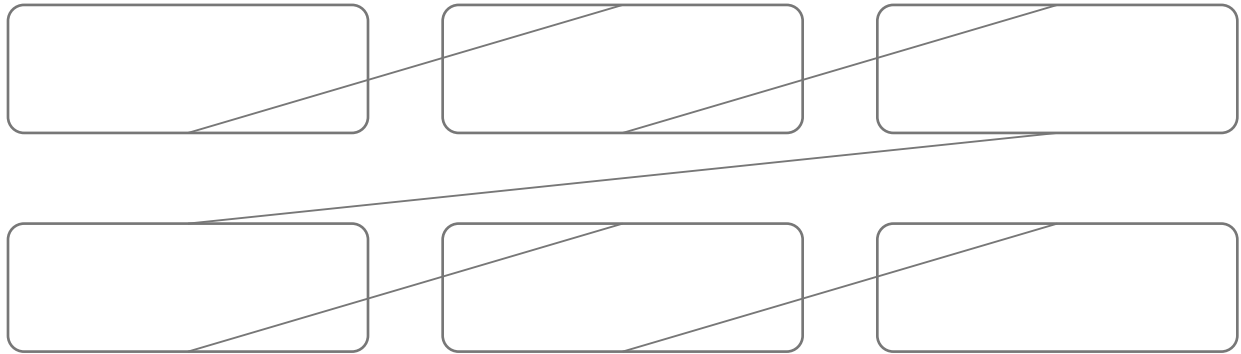


Fig. 3.4: Student process flow.

3.14 Mentor DFD

The Mentor workflow allows the mentor to log in, maintain professional information, receive mentorship requests, manage sessions, communicate with students, and view relevant job interview schedules when the mentor is a candidate.

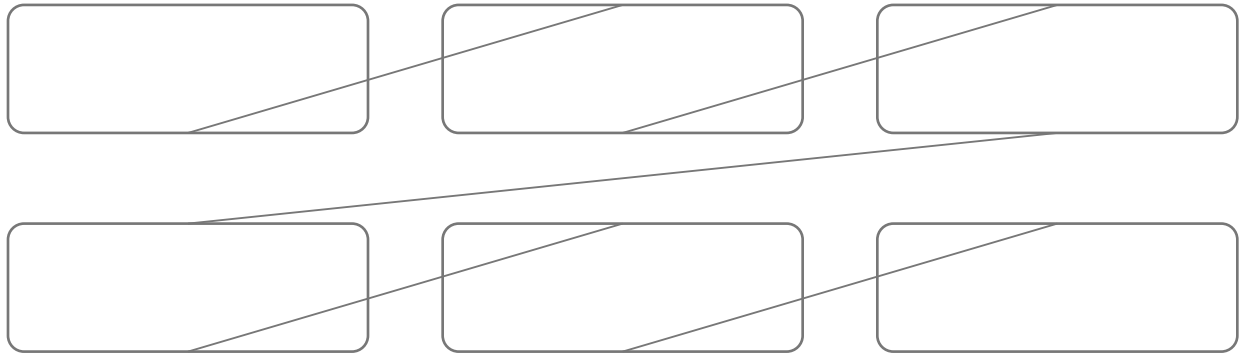


Fig. 3.5: Mentor process flow.

3.15 Recruiter DFD

The Recruiter workflow starts with authentication and profile access. The recruiter creates job posts, views applicants, reviews candidate information, shortlists candidates, and schedules online interviews with a future date, time, and meeting link.

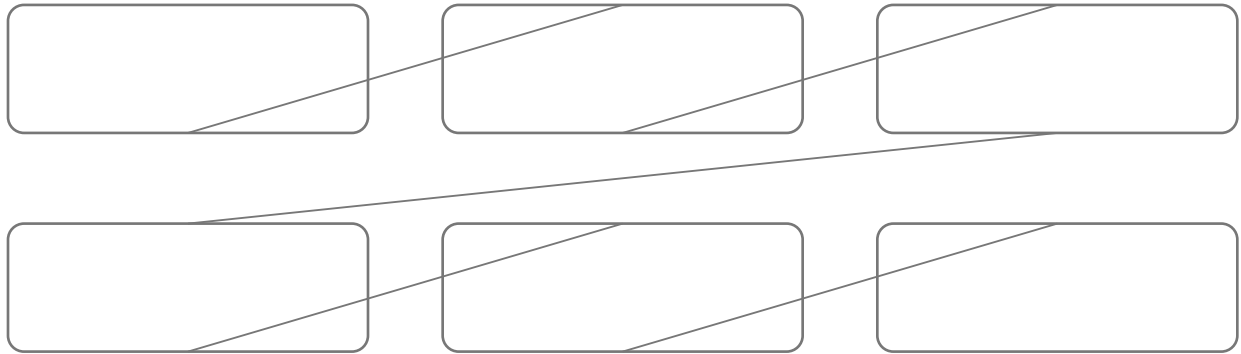


Fig. 3.6: Recruiter process flow.

3.16 ER Diagram

The Entity Relationship structure of CareerConnect contains User, Job, Application, Mentorship Request, Session, Review, Message, Notification, and Interview entities. Users participate in different workflows according to their role. Jobs are created by recruiters and applications connect candidates with jobs. Interviews connect recruiters, candidates, and jobs.

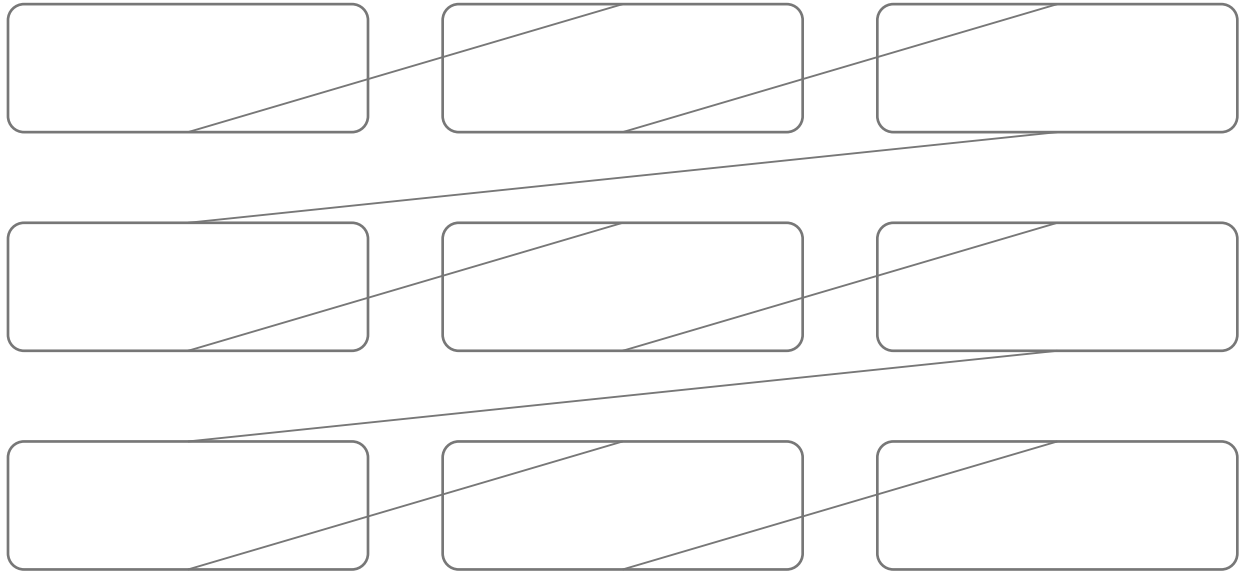


Fig. 3.7: Conceptual ER structure.

3.17 Database Design

Major MongoDB collections used by the application include:

Collection / Model	Purpose
User	Stores name, email, password, role and profile information.
Job	Stores job title, company, location, salary, type, skills and description.
Application	Connects applicants with jobs and stores application status.
MentorshipRequest	Stores student-mentor requests and request status.
Session	Stores mentorship session information and status.
Review	Stores feedback and 1-7 rating.
Message	Stores communication between users.
Notification	Stores alerts for important user actions.
Interview	Stores recruiter, candidate, job, date/time, meeting link, message and status.

4. System Design

4.1 Modules

- Authentication Module – registration and login.
- Student Module – profile, mentors, mentorship, jobs and interviews.
- Mentor Module – professional profile, requests, sessions and interviews.
- Recruiter Module – job posts, applicants, shortlisting and interviews.
- Communication Module – messages and notifications.
- Review Module – feedback and rating.
- Interview Module – online interview scheduling and candidate access.

4.2 Architecture

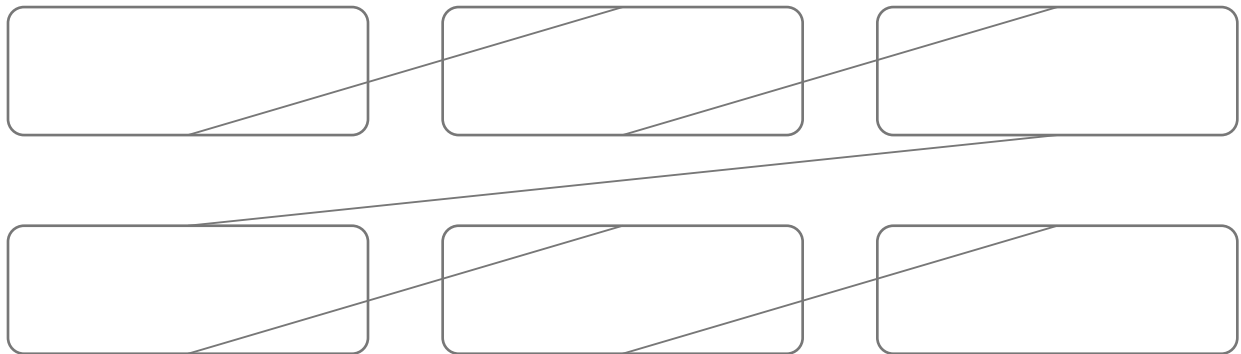


Fig. 4.1: CareerConnect system architecture.

4.3 User Module

The User Module is responsible for account registration, login, profile management, and role-based access. After successful authentication, the application loads the appropriate dashboard according to the user's role.

Function	Description
Registration	Creates a new account with name, email, password and role.
Login	Validates credentials and opens the relevant dashboard.
Profile	Allows users to add education, skills, experience, headline and bio.
Logout	Ends the current frontend session.

4.4 Student Panel Design

The Student Dashboard provides access to profile information, mentor discovery, mentorship requests, sessions, messages, jobs, applications, and scheduled job interviews. The student can open a mentor profile, request mentorship, and later view the status of requests.

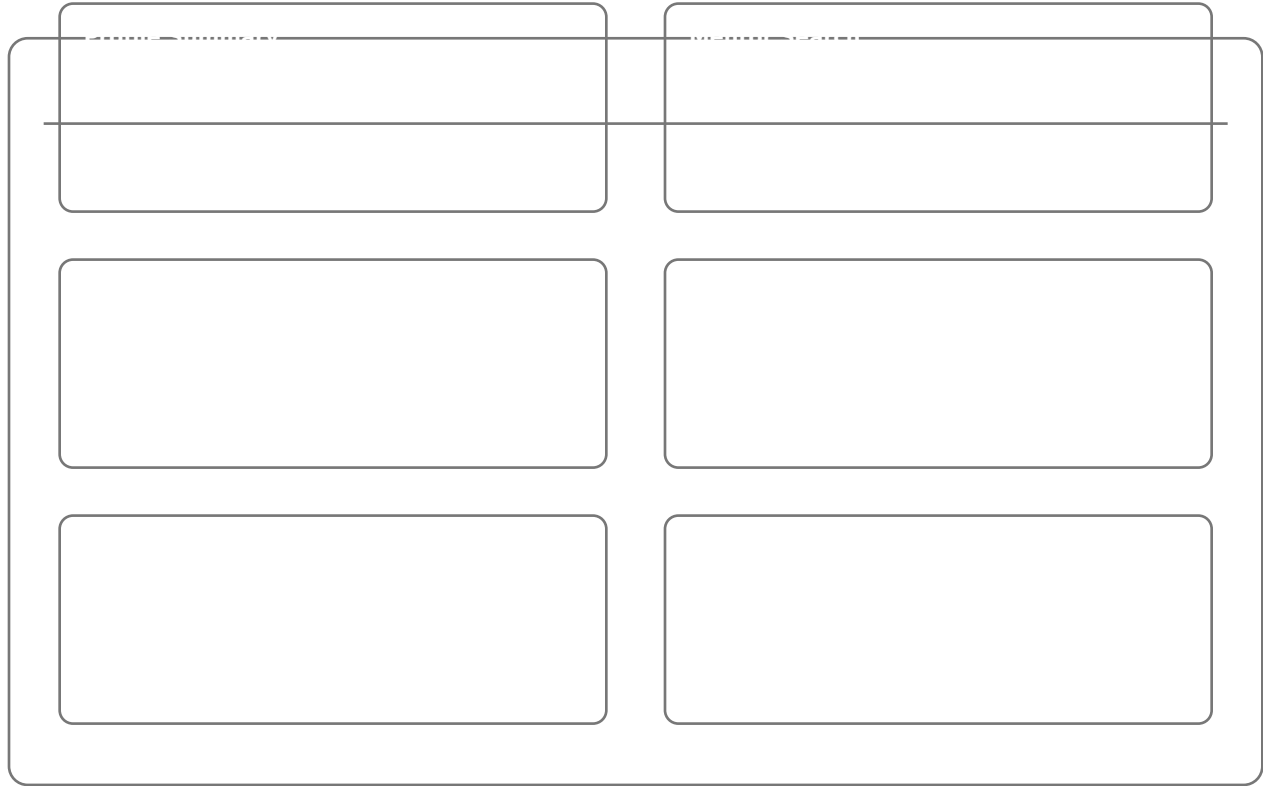


Fig. 4.2: Student dashboard layout.

4.5 Mentor Panel Design

The Mentor Dashboard focuses on professional information, mentorship requests, sessions, messages, notifications, and job interviews. A mentor can update skills, experience, education, headline, and bio and can respond to student requests.

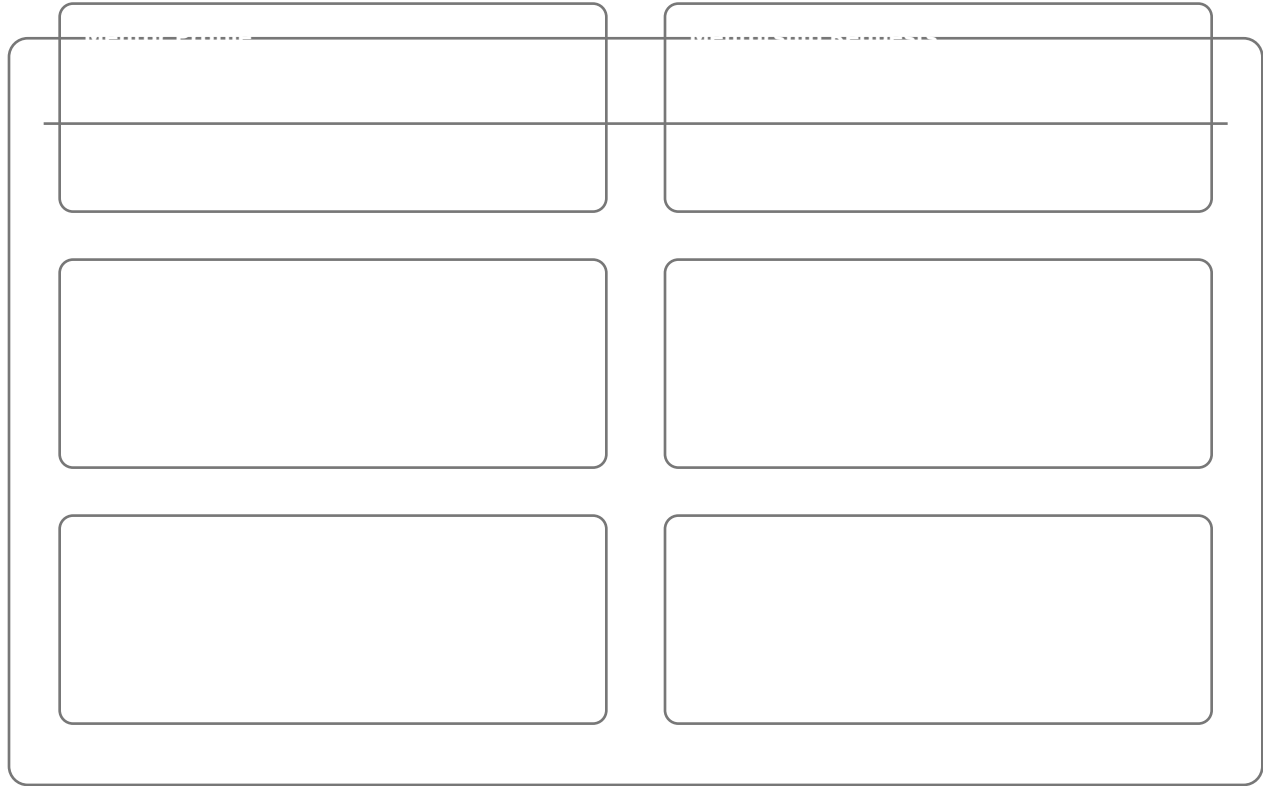


Fig. 4.3: Mentor dashboard layout.

4.6 Recruiter Panel Design

The Recruiter Dashboard allows job providers to create job posts and manage applications. For a selected applicant, the recruiter can open candidate details, shortlist the candidate, and schedule an online interview by entering a future date/time and meeting link.

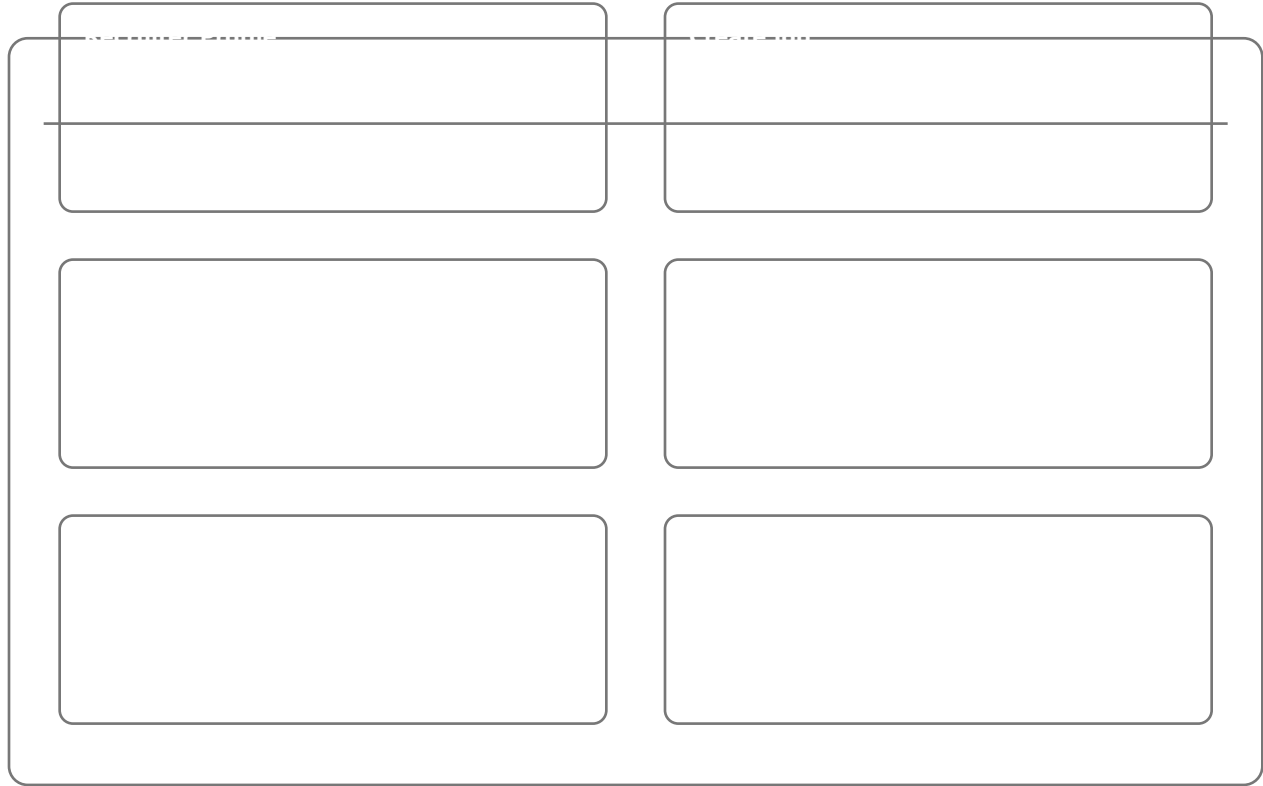


Fig. 4.4: Recruiter dashboard layout.

4.7 Job Application Workflow

A recruiter creates a job containing title, company, location, salary, job type, experience, skills, description, and other relevant information. Students or eligible users can view the job and submit an application. The recruiter can then view applicants and take recruitment actions.

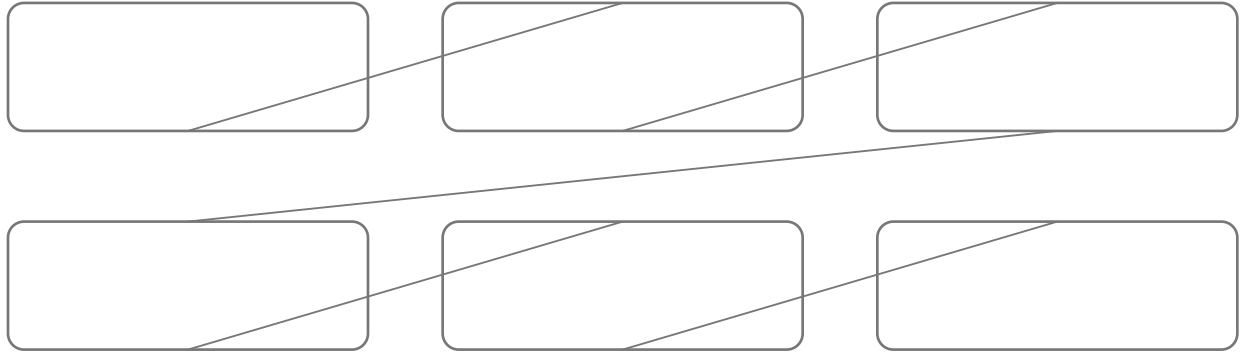


Fig. 4.5: Job application workflow.

4.8 Mentorship Workflow

The mentorship workflow begins when a student searches for a mentor and opens the mentor's profile. The student sends a mentorship request. The mentor can review the request and respond. Sessions can then be requested or managed, followed by feedback and rating.

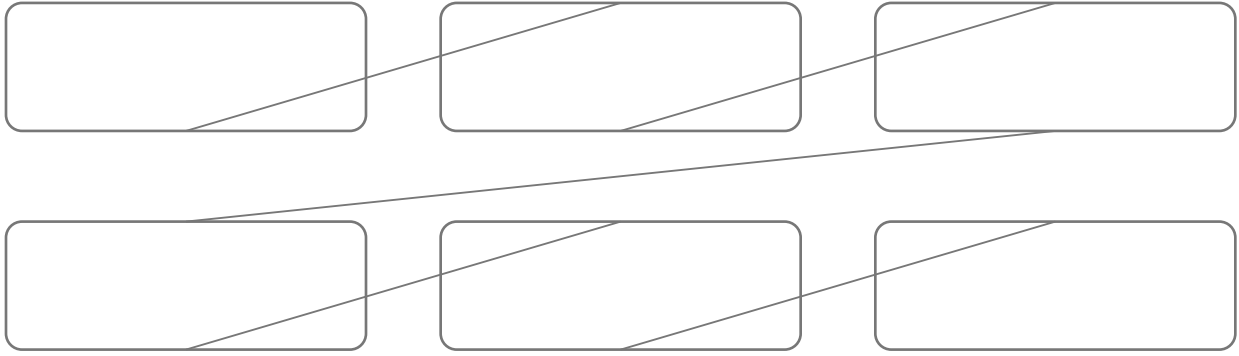


Fig. 4.6: Mentorship workflow.

4.9 Interview Scheduling Workflow

Recruiters manually schedule online interviews for selected candidates. The form requires a valid future interview date/time and a meeting URL. The backend validates the recruiter, application, job, candidate, date, and meeting link before storing the interview. The candidate dashboard displays scheduled interviews and provides a Join Interview link.

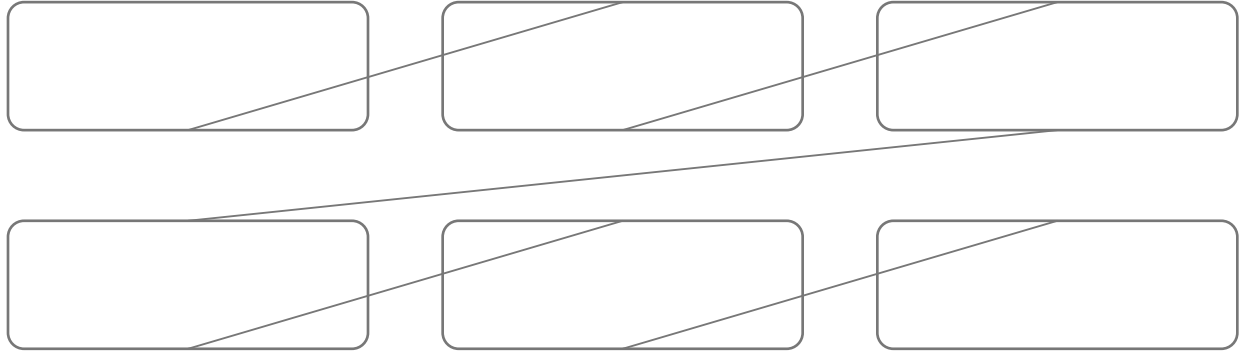


Fig. 4.7: Interview scheduling workflow.

4.10 Procedural Design - Authentication

The login procedure accepts email and password, sends the information to the backend, validates the credentials, and returns the authenticated user information. The frontend stores the required session information and redirects the user to the appropriate role dashboard.

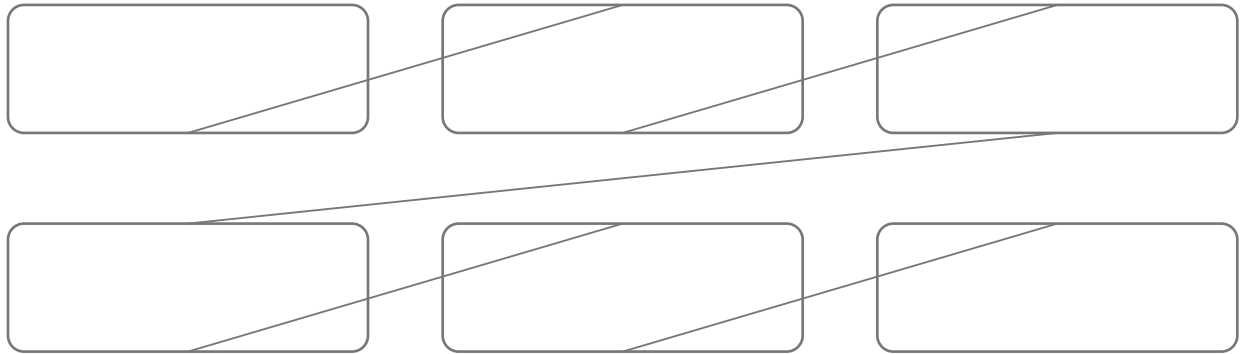


Fig. 4.8: Authentication flow.

4.11 Procedural Design - Interview Validation

Interview scheduling contains validation to prevent incomplete or invalid records. The backend checks the recruiter ID and application ID, confirms the application and job, checks recruiter ownership, verifies the candidate, validates the future date/time, and validates the HTTP/HTTPS meeting URL. Google Meet links are normalized so that a missing slash after the domain is corrected before saving.

Validation	Purpose
Required fields	Prevents incomplete scheduling requests.
ObjectId validation	Prevents invalid database identifiers.
Application / Job	Ensures the selected application and job exist.
Recruiter ownership	Prevents scheduling interviews for another recruiter's job.
Future date/time	Prevents past interview dates.
Meeting URL	Requires a valid HTTP/HTTPS meeting link.

4.12 Security and Data Validation

- Role-specific routes and UI controls reduce access to unrelated functions.
- MongoDB ObjectId validation is used where identifiers are received through APIs.
- Required fields are checked before creating important records.
- Interview scheduling verifies recruiter ownership of the job.
- Meeting URLs are parsed and restricted to HTTP/HTTPS protocols.
- Sensitive credentials should not be exposed in frontend source code.
- Production deployment should add HTTPS, secure cookies/tokens, stronger password hashing, rate limiting, and environment-variable protection.

4.13 User Interface Principles

- Consistent navigation between dashboard sections.
- Clear action buttons for profile, mentorship, jobs, applications, and interviews.
- Feedback messages after successful or failed operations.
- Readable cards and tables for structured information.
- Reusable React components to reduce duplication.

SCREENSHOTS / INTERFACE VIEWS

The following pages document the major interface areas of CareerConnect. They represent the screen structure and user workflow of the developed system.

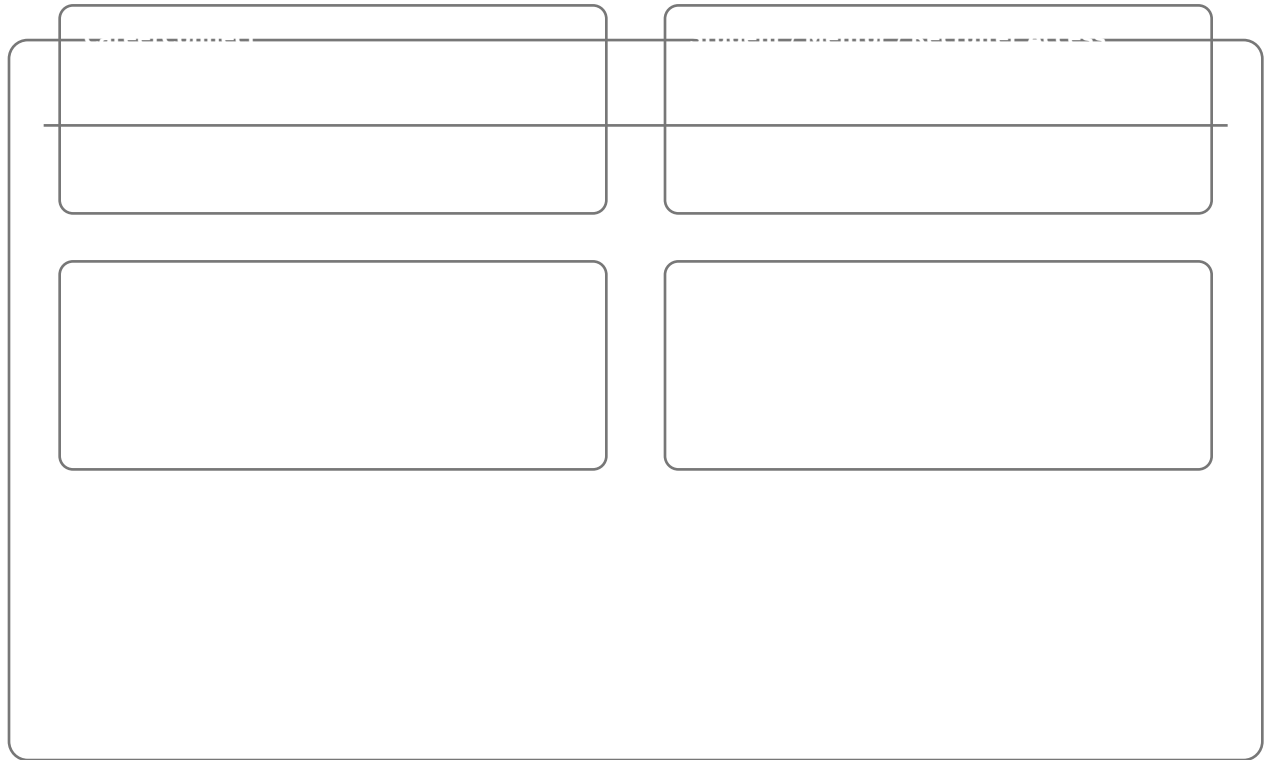


Fig. 5.1: Home / landing interface view.

Student Dashboard

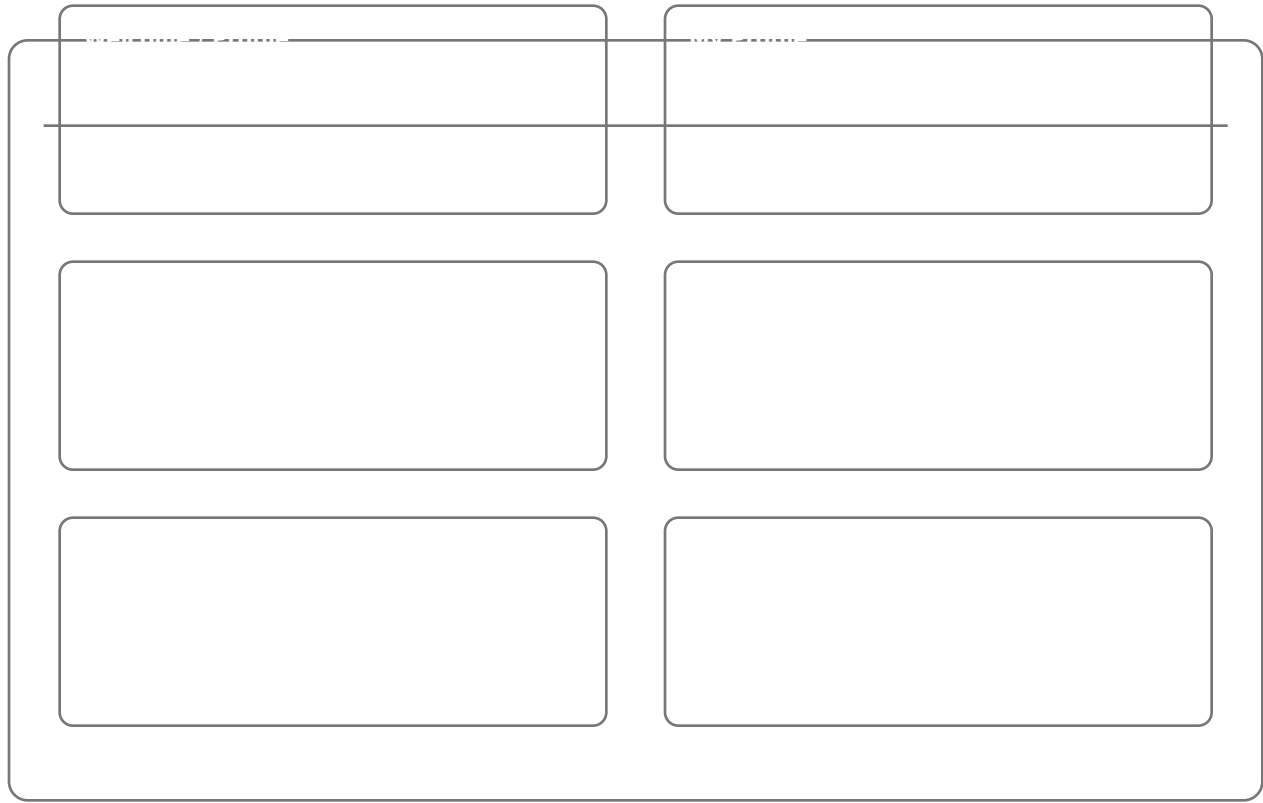


Fig. 5.2: Student dashboard interface view.

Student Profile and Mentor View

Fig. 5.3: Profile and mentor discovery interface view.

Mentor Dashboard

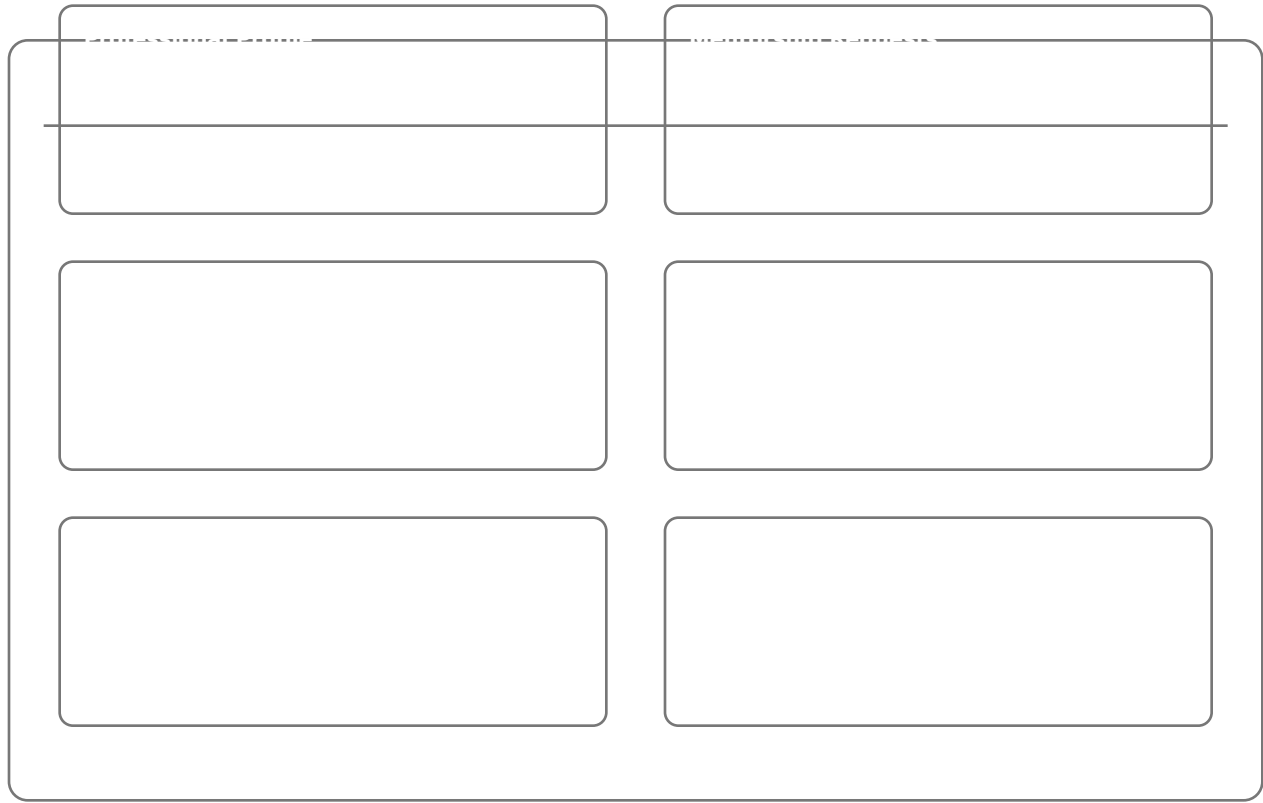


Fig. 5.4: Mentor dashboard interface view.

Recruiter Dashboard

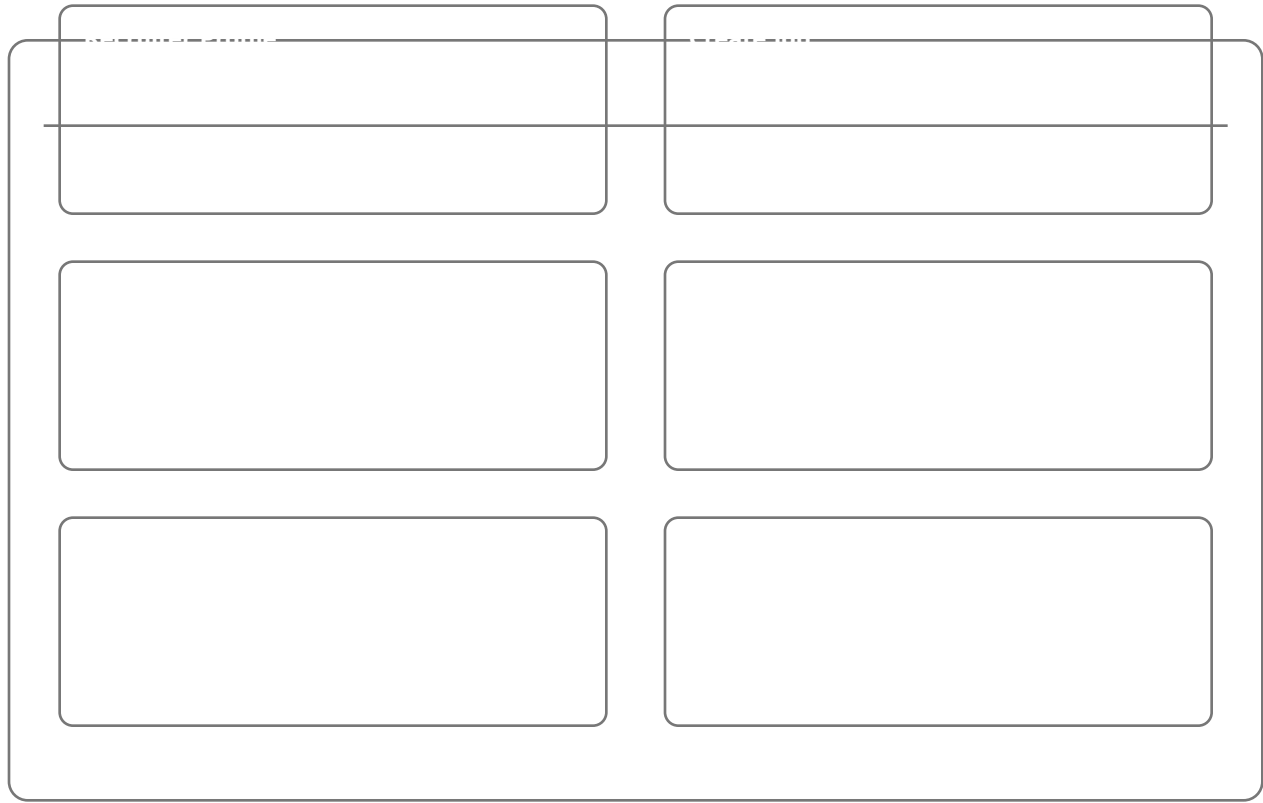


Fig. 5.5: Recruiter dashboard interface view.

Job Application and Applicant Management

Fig. 5.6: Job application and applicant management interface view.

Online Interview Scheduling and Candidate View

The diagram illustrates a user interface for online interview scheduling and candidate view. It consists of a large container with rounded corners, which holds a grid of six smaller rounded rectangular panels arranged in three rows and two columns. A horizontal line is drawn across the top of the grid, separating the top row from the middle and bottom rows. The panels are currently empty, representing a wireframe or placeholder for content.

Fig. 5.7: Online interview scheduling and candidate interface view.

Coding

6.1 Frontend Project Structure

```
frontend/  
├── src/  
│   ├── components/  
│   ├── pages/  
│   ├── assets/  
│   ├── App.jsx  
│   └── main.jsx  
└── package.json
```

The React frontend contains reusable components, page-level dashboards, routing, and API communication. Axios is used for requests to the Express backend.

6.2 Backend Project Structure

```
backend/  
├── models/  
├── routes/  
├── server.js  
├── .env  
└── package.json
```

The backend separates models and route handlers so that database operations and API endpoints remain maintainable.

6.3 Express Server Configuration

```
const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");
require("dotenv").config();

const interviewsRoute = require("./routes/interviews");

const app = express();

app.use(cors());
app.use(express.json());

app.use("/api", interviewsRoute);

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log("MongoDB Connected Successfully!"))
  .catch(err => console.log("MongoDB Connection Error:", err.message));

app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});
```

The server initializes Express, enables CORS and JSON parsing, mounts API routes, connects to MongoDB, and starts the application server.

6.4 User Model

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: String,
    email: String,
    password: String,
    role: {
      type: String,
      enum: ["student", "mentor", "recruiter"],
      required: true,
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);
```

The User model stores the common account identity and role. Profile-related information can be extended through the profile workflow.

6.5 Interview Model

```
const interviewSchema = new mongoose.Schema(  
  {  
    recruiter: {  
      type: mongoose.Schema.Types.ObjectId,  
      ref: "User",  
      required: true,  
    },  
    candidate: {  
      type: mongoose.Schema.Types.ObjectId,  
      ref: "User",  
      required: true,  
    },  
    job: {  
      type: mongoose.Schema.Types.ObjectId,  
      ref: "Job",  
      required: true,  
    },  
    jobTitle: String,  
    candidateEmail: String,  
    interviewDate: Date,  
    meetingLink: String,  
    message: String,  
    status: {  
      type: String,  
      enum: ["scheduled", "completed", "cancelled"],  
      default: "scheduled",  
    },  
  },  
  { timestamps: true }  
);
```

The Interview model links recruiter, candidate, and job and stores the scheduling information required by the online interview workflow.

6.6 Interview Scheduling API

```
router.post("/interviews", async (req, res) => {
  const {
    recruiterId,
    applicationId,
    interviewDate,
    meetingLink,
    message,
  } = req.body;

  // validate IDs, application, job and candidate
  // validate future interview date
  // validate HTTP/HTTPS meeting link

  const interview = new Interview({
    recruiter: recruiterId,
    candidate: candidate._id,
    job: job._id,
    jobTitle: job.jobTitle,
    candidateEmail: candidate.email,
    interviewDate: new Date(interviewDate),
    meetingLink: parsedLink.toString(),
    message: message || "",
  });

  await interview.save();

  res.status(201).json({
    success: true,
    message: "Online interview scheduled successfully!",
    interview,
  });
});
```

The scheduling route validates the request and creates an Interview document only after the related records have been verified.

6.7 Candidate Interview API

```
router.get("/interviews/candidate/:candidateId", async (req, res) => {
  const { candidateId } = req.params;

  const interviews = await Interview.find({
    candidate: candidateId,
    status: "scheduled",
  })
    .populate("recruiter", "name email")
    .populate("job", "jobTitle company")
    .sort({ interviewDate: 1 });

  return res.status(200).json({
    success: true,
    interviews,
  });
});
```

This API returns scheduled interviews for a candidate and populates recruiter and job information for display on the dashboard.

6.8 React Interview Display

```
const fetchInterviews = async () => {
  try {
    setInterviewsLoading(true);

    const response = await axios.get(
      `http://localhost:5000/api/interviews/candidate/${user.id}`
    );

    setInterviews(response.data.interviews || []);
  } catch (error) {
    console.log("Fetch Interviews Error:", error);
  } finally {
    setInterviewsLoading(false);
  }
};

<a
  href={interview.meetingLink}
  target="_blank"
  rel="noreferrer"
>
  ☐ Join Interview
</a>
```

The dashboard fetches scheduled interviews and displays a link that opens the stored online meeting URL in a new browser tab.

Testing

7.1 Testing Strategy

Testing was performed at multiple levels to verify individual functions, API integration, database operations, and complete user workflows.

Test Type	Purpose	Examples
Unit Testing	Check individual functions/components.	Form validation, UI actions
Module Testing	Check a group of related functions.	Mentorship, jobs, interviews
Integration Testing	Check frontend-backend communication.	Axios requests and API responses
System Testing	Check complete workflows.	Register → login → dashboard
Acceptance Testing	Verify the system against project requirements.	Student, mentor and recruiter workflows

7.2 Sample Test Cases

TC	Test Case	Expected Result
TC01	Register new user	User record is created.
TC02	Login with valid credentials	Correct dashboard opens.
TC03	Update profile	Profile data is saved.
TC04	Send mentorship request	Request is stored and shown.
TC05	Create job	Job is displayed to eligible users.
TC06	Apply for job	Application is stored.
TC07	Schedule interview	Interview is saved with scheduled status.
TC08	Open candidate interviews	Scheduled interview appears.
TC09	Join interview	Meeting link opens in browser.

Report

8.1 Project Development Report

CareerConnect was developed as a full-stack MERN web application. The project was divided into frontend, backend, database, and integration activities. The frontend provides role-based pages and dashboards, while the backend exposes REST APIs for authentication, profiles, mentorship, jobs, applications, reviews, sessions, messages, notifications, and interviews.

8.2 Development Outcome

- Student dashboard and profile workflow implemented.
- Mentor dashboard and professional profile workflow implemented.
- Recruiter job management and applicant workflow implemented.
- Mentorship request and session workflow implemented.
- Reviews and rating support implemented.
- Online interview scheduling implemented with future date/time validation.
- Candidate interview listing and Join Interview link implemented.
- MongoDB integration established for persistent data.

8.3 Project Significance

The project demonstrates how a modern full-stack web application can combine career guidance and recruitment workflows in one platform. Its modular structure also makes it suitable for future improvements such as advanced matching, analytics, and hosted deployment.

6. Future Scope

- AI-based mentor and job recommendations based on skills and interests.
- Advanced search and filtering for mentors and jobs.
- Real-time chat using WebSocket technology.
- Calendar integration for mentorship sessions and interviews.
- Automated interview reminders and notifications.
- Resume upload, parsing, and skill extraction.
- Recruiter analytics for applications and hiring stages.
- Video-session integration for mentorship.
- Mobile application for Android and iOS.
- Cloud deployment with scalable storage and authentication.

Conclusion

CareerConnect provides an integrated platform for students, mentors, and recruiters. It organizes professional profiles, mentorship, sessions, reviews, jobs, applications, shortlisting, and online interviews into role-specific workflows. The MERN architecture provides a clear separation between the React frontend, Express/Node.js backend, and MongoDB database. The project demonstrates the practical use of REST APIs, database models, role-based workflows, and reusable user interfaces. With future enhancements such as intelligent recommendations, real-time communication, analytics, and cloud deployment, the platform can be extended into a larger career management ecosystem.

Bibliography

- MongoDB Documentation – Database and MongoDB data modeling concepts.
- Express.js Documentation – REST API and server-side routing.
- React Documentation – Components, state, props, and frontend development.
- Node.js Documentation – JavaScript runtime and server-side development.
- MDN Web Docs – HTML, CSS, JavaScript, HTTP, and browser APIs.

References

- React Official Documentation – <https://react.dev/>
- Node.js Official Website – <https://nodejs.org/>
- Express.js Official Website – <https://expressjs.com/>
- MongoDB Official Website – <https://www.mongodb.com/>
- MDN Web Docs – <https://developer.mozilla.org/>

End of Project Report